Université Paris Cité, Campus Saint-Germain-des-Près
UFR de Mathématiques et Informatique
45, rue des Saints-Pères 75006 Paris

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Algorithmique et Programmation 2

Correction du contrôle terminal du 15 mai 2024 — durée: 1h30

Tout document interdit. Les calculatrices et les téléphones portables, même prévus à titre d'horloge, sont également interdits.

Tous les exercices sont indépendants. On précisera bien la signature (type des entrées/sorties) de chaque fonction (Python ou pseudo-code) proposée. Sur la note globale, 1 point sera attribué en fonction de la proportion de signatures correctes.

Exercice 1. Pour chaque ligne (indépendante) d'instructions ci-dessous, donner le contenu de la variable `x` après exécution de la ligne considérée si l'exécution de la ligne ne provoque pas d'erreur, ou expliquer la nature de l'erreur dans le cas contraire. **Expliquer votre réponse dans les deux cas (une réponse correcte sans explication ne sera pas comptabilisée).**

```
1  S = {-1,0,1} - {1}; x = (S <= {0})
2  S = {-1,0,1} + {1}; x = (S >= {0})
3  x = frozenset(range(3,10)) & frozenset(range(7,-2,-3))
4  x = {n**2 for n in range(3)}; x.remove(9)
5  x = {}; x.add((0,))
6  x = ({(1,2)} | {(2,1)}) - {1,2}
7  S = set("abc"); S.add("abc"); x = len(S)
8  D = {1:"a"}; D["a"] = 2; x = D[D[1]]
9  D = {1:3, 2:5, 3:0}; x = sum(z*D[z] for z in D)
10 D = {z**2:z for z in range(10)}; x = D.get(54,36)
```

(1) Ligne 1: S est l'ensemble (de type `set`) $\{-1, 0\}$, qui n'est pas inclus dans l'ensemble $\{0\}$, donc `x=False`.
(2) Ligne 2: erreur car l'opérateur `+` n'est pas défini entre 2 objets de type `set`.
(3) Ligne 3: `x` est l'ensemble (de type `frozenset`) obtenu par intersection de $\{3, 4, 5, 6, 7, 8, 9\}$ et $\{7, 4, 1\}$, donc `x=frozenset({4,7})`.
(4) Ligne 4: `x` est l'ensemble (de type `set`) $\{0, 1, 4\}$. Cet ensemble ne contient pas 9 donc `x.remove(9)` provoque une erreur.
(5) Ligne 5: `x` est un dictionnaire vide, or la méthode `add()` n'est pas définie sur un dictionnaire, donc erreur.
(6) Ligne 6: `x` est un ensemble (de type `set`) égal à $\{(1, 2), (2, 1)\}$ (union de deux ensembles à un élément, chaque élément étant un tuple à deux éléments; la fin de la ligne `-{1,2}` n'a aucun effet car 1 et 2 ne font pas partie de l'ensemble formé par l'union).
(7) Ligne 7: On ajoute à l'ensemble (`set`) initial `{"a","b","c"}` l'élément `"abc"`, donc à la fin `S = {"a","b","c","abc"}` donc `x=4`.
(8) Ligne 8: `x = D[D[1]] = D["a"] = 2`.
(9) Ligne 9: `x = 1.3 + 2.5 + 3.0 = 13`.
(10) Ligne 10: `D` est le dictionnaire `{0:0, 1:1, 4:2, 9:3, 16:4, 25:5, ..., 81:9}` (les clés sont les carrées des entiers de 0 à 9 inclus, les valeurs les racines carrées des clés). Comme 54 n'est pas le carré d'un entier, ce n'est pas une clé de `D` donc `D.get(54,36)` retourne son deuxième argument et `x=36`.

Exercice 2. Pour chaque fonction ci-dessous, proposez un algorithme **sous forme de pseudo-code** (les codes Python ne seront pas acceptés ici).

(1) La fonction récursive f_1 prend en entrée la racine d'un arbre, et retourne le nombre de nœuds (y compris la racine) qui ont exactement deux enfants.

```
fonction f1(N)
    // noeud -> entier
    // retourne le nombre de noeuds qui ont exactement deux enfants
    compte <- 0
    si taille(enfants(N)) = 2
        compte <- 1
    pour tout enfant E de N
        compte <- compte + f1(E)
    retourner compte
```

(2) La fonction récursive f_2 prend en entrée la racine d'un arbre dont les nœuds sont étiquetés par des nombres, et retourne le nombre de nœuds qui ont la même valeur d'étiquette que leur parent.

```
fonction f2(N)
    // noeud -> entier
    // retourne le nombre de noeuds qui ont la même étiquette que leur parent
    compte <- 0
    pour tout enfant E de N
        si étiquette(E) = étiquette(N)
            compte <- compte + 1
        compte <- compte + f2(E)
    retourner compte
```

(3) La fonction f_3 prend entrée deux files A et B, et retourne une file C formée par les éléments de A puis ceux de B.

```
fonction f3(A, B)
    // file x file -> file
    // retourne une file constituée des éléments de A puis de B
    C <- file vide
    tant que A est non vide
        extraire un élément x de A
        ajouter x à C
    tant que B est non vide
        extraire un élément x de B
        ajouter x à C
    retourner C
```

(4) La fonction f_4 prend en entrée un graphe G non orienté et non vide et retourne un sommet (quelconque) de G parmi les sommets de degré maximal (on rappelle que le degré d'un sommet est son nombre de voisins).

```
fonction f4(G)
    // graphe -> sommet
    // retourne un sommet de degré maximal
    dmax <- -1
    pour tout sommet s de G
        d <- nombre de voisins de s
        si d > dmax
            dmax <- d
            t <- s
    retourner t
```

Exercice 3. Traduire le pseudo-code ci-dessous en langage Python. Dans ce pseudo-code, on suppose que les chaînes de caractères sont indexées à partir de l'indice 0, et pour une chaîne de caractères s de taille n, le caractère d'indice k (avec $0 \leqslant k \leqslant n-1$) est désigné par $s(k)$.

fonction recherche(s_1, s_2)
// chaîne de caractères $\times$ chaîne de caractères $\rightarrow$ booléen
// teste si la chaîne s_1 est présente dans la chaîne s_2
$n_1 \leftarrow$ taille de s_1
$n_2 \leftarrow$ taille de s_2
pour $i = 0, 1, ..., n_2 - 1$
    $k \leftarrow 0$
    tant que $k < n_1$ et $i + k < n_2$ et $s_1(k) = s_2(i + k)$
        $k \leftarrow k + 1$
    si $k = n_1$
        retourner vrai
retourner faux

La traduction ne pose pas de problème particulier:

```python
def recherche(s1, s2):
    ''' str x str -> bool
    teste si la chaîne s1 est présente dans la chaîne s2
    '''
    n1 = len(s1)
    n2 = len(s2)
    for i in range(n2):
        k = 0
        while k<n1 and i+k<n2 and s1[k]==s2[i+k]:
            k = k+1
        if k==n1:
            return True
    return False
```

Exercice 4. Proposer une implémentation Python de chaque fonction ci-dessous.

(1) La fonction `g1` prend en entrée un dictionnaire `D` dont les valeurs sont supposées immutables et distinctes (on ne demande pas de vérifier cette condition dans le code) et retourne un dictionnaire "inverse" `E` qui vérifie `E[y]=x` pour tous `(x,y)` tels que `y=D[x]`.
Par exemple, `g1({1:5, 2:4, 3:0})` doit retourner `{5:1, 4:2, 0:3}`.

Un simple parcours de `D` suffit à définir `E` en inversant clés et valeurs.

```
def g1(D):
    ''' dict -> dict
    retourne le dictionnaire "inverse" de D
    '''
    E = dict()
    for x in D:
        E[D[x]] = x
    return E
```

(2) La fonction génératrice `g2` qui prend en entrée une liste d'entiers L non vide et un entier naturel n, et délivre dans l'ordre croissant tous les entiers $k \in \{0, 1, \ldots, n\}$ qui sont multiples d'exactement un élément de L. Par exemple, `list(g2([3,7,6],15))` doit retourner la liste `[3,7,9,14,15]`.

Il suffit de parcourir chaque entier $k \in \{0, 1, \ldots, n\}$ et de compter le nombre d'éléments de `L` qui le divisent; si ce nombre est un, alors il faut délivrer k.

```
def g2(L, n):
    ''' list x int -> generator
    retourne un générateur des entiers k, 0<=k<=n
    qui sont multiple d'exactement un élément de L
    '''
    for k in range(n+1):
        nombre = 0
        for x in L:
            if k%x==0:
                nombre += 1
        if nombre==1:
            yield k
```

On peut racourcir sensiblement le code précédent en utilisant la fonction `sum()` (qui, lorsqu'on lui demande de sommer des booléens, compte le nombre de valeurs `True`):

```
def g2(L, n):
    ''' list x int -> generator
    retourne un générateur des entiers k, 0<=k<=n
    qui sont multiple d'exactement un élément de L
    '''
    for k in range(n+1):
        if sum(k%x==0 for x in L)==1:
            yield k
```

(3) La fonction <u>récursive</u> `g3` qui prend en entrée un entier naturel n, et renvoie le terme u_n de la suite définie par $u_0 = 1$ et la relation de récurrence

$$\forall n \in \mathbb{N}, \quad u_{n+1} = \begin{cases} u_n^2 & \text{si } n \text{ est pair,} \\ u_n + n & \text{sinon.} \end{cases}$$

Par exemple, `g3(4)` doit retourner 7.

Attention à ne pas se tromper dans les indices, car dans le code il faut calculer u_n en fonction de u_{n-1}, et non u_{n+1} en fonction de u_n. Il est sage de commencer par réécrire la récurrence sous la "bonne" forme

$$\forall n \in \mathbb{N}^*, \quad u_n = \begin{cases} u_{n-1}^2 & \text{si } n-1 \text{ est pair,} \\ u_{n-1} + n - 1 & \text{sinon.} \end{cases}$$

```
def g3(n):
    ''' int -> int
    retourne la valeur de u(n)
    '''
    if n==0:
        return 1
    if (n-1)%2==0:
        return g3(n-1)**2
    return g3(n-1)+n-1
```

(4) La fonction génératrice <u>récursive</u> `g4` prend en entrée un entier $n \geqslant 0$, et délivre les coefficients du binôme $\binom{n}{k}$ pour $k = 0, 1, \ldots, n$. On pourra utiliser la relation

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Par exemple, `list(g4(4))` doit retourner la liste `[1,4,6,4,1]`.

Pour $n = 0$, il y a seulement la valeur 1 à délivrer (condition d'arrêt de la récursivité).
Pour $n > 0$, on fabrique les coefficients à partir de ceux délivrés pour $n - 1$:

- on commence par délivrer 1
- on parcours les coefficients délivrés pour n-1, et on délivre leur sommes 2 à 2
- on termine en délivrant 1

```
def g4(n):
    ''' int -> generator
    délivre les coefficients du binôme k parmi n'' pour k=0,1,...,n
    '''
    if n==0:
        yield 1
    else:
        yield 1
        x = 0
        for y in g4(n-1):
            if x!=0:
                yield x+y
            x = y
        yield 1
```

Exercice 5. On souhaite manipuler avec un classe Python des polynômes de degré au plus 2. Écrire un code Python permettant de définir une telle classe `Trinome` avec les fonctionnalités suivantes :

(1) `Trinome(a,b,c)` crée une instance de la classe correspondant au polynôme $aX^2 + bX + c$

(2) si `P` et `Q` sont deux instances de la classe `Trinome`, alors `P+Q` retourne une nouvelle instance de la classe correspondant à la somme des deux polynômes associés

(3) si `P` est une instance de la classe `Trinome`, alors `P.derive()` retourne une nouvelle instance de la classe correspondant à la dérivée du polynôme représenté par `P`

On définit une classe `Trinome` à l'aide du mot réservé `class`, et:

- La création d'une instance se fait avec la méthode spéciale `__init__()`, qui se contente ici de recopier les arguments `a,b,c` dans des attributs de la classe (ici on garde les mêmes noms mais ce n'est pas une obligation).
- L'addition de deux instances de la classe se fait avec la méthode spéciale `__add__()`. On retourne simplement une nouvelle instance obtenue en sommant terme à terme les coefficients des deux polynômes (attributs `a,b,c`).
- Pour la dérivation, on crée une nouvelle instance qui code le polynôme $0.X^2 + 2a.X + b$, puisque c'est la dérivée de $a.X^2 + b.X + c$.

```python
class Trinome():
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def __add__(P, Q):
        return Trinome(P.a+Q.a, P.b+Q.b, P.c+Q.c)

    def derive(self):
        return Trinome(0, 2*self.a, self.b)
```